



UNIVERSITÀ DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Informatica

ELABORATO FINALE

UN CONNETTORE PER LO STORAGE DEI DATI PER LA PIATTAFORMA ILOG

Supervisore

Fausto Giunchiglia

Co-supervisore

Leonardo Javier Malcotti

Laureando

Alessia Angeli

Anno accademico 2024/2025

Ringraziamenti

Vorrei ringraziare il Professor Fausto Giunchiglia per avermi dato l'opportunità di lavorare su questa tesi, permettendomi di approfondire un tema stimolante e di migliorare le mie capacità di analisi e di ricerca. Un grazie sincero anche all'assistente Leonardo Malcotti, per il supporto e la disponibilità dimostrata, soprattutto nei momenti in cui ho avuto difficoltà con la tesi. Infine, un ringraziamento speciale alla mia famiglia, che mi ha sempre sostenuto con pazienza e affetto durante tutto il mio percorso universitario.

Indice

Sommario	3
1 Introduzione	3
2 Stato dell'arte	5
2.1 Storia delle Tecnologie e dei Metodi per Data Warehouse e Data Lakes	5
2.1.1 La Nascita dei Data Warehouse	5
2.1.2 L'Avvento dei Data Lakes e di Hadoop	5
2.1.3 Il Ruolo del Cloud e la Convergenza tra Data Lakes e Data Warehouse	6
2.1.4 I Data Lakes: Versatilità e Sfide	6
2.1.5 La Nuova Era: Integrazione tra Data Lakes e Data Warehouse	7
2.1.6 Metodologie e Nuove Pratiche	7
2.1.7 Conclusioni	7
2.2 Architettura Lambda e Delta	8
2.2.1 Architettura Lambda	8
2.2.2 Architettura Delta	8
2.2.3 Verso una Unified Architecture	9
2.2.4 Conclusioni	9
2.3 Dallo Stato dell'Arte alla Soluzione: Delta Lake	10
2.3.1 Le Funzionalità Chiave di Delta Lake	10
2.3.2 Come Funziona Delta Lake	11
2.3.3 La Soluzione Completa: Delta Lake come Piattaforma di Big Data	11
3 Architettura Corrente	12
3.1 Diagramma sistema attuale	12
3.1.1 Kubernetes	12
3.1.2 Apache Kafka	13
4 Requisiti	14
4.1 Requisiti Funzionali	14
4.2 Requisiti Non Funzionali	15
5 Soluzione proposta	15
5.1 Architettura e flusso dei dati	15
5.2 Tecnologie aggiunte	16
5.2.1 Delta Lake	16
5.2.2 Apache Flink	17
5.2.3 MinIO	17
5.2.4 Apache Parquet	19
6 Implementazione	21
6.1 Kafka Delta Sink	21
7 Conclusioni	25

Sommario

Negli ultimi anni, gestire in modo efficace i dati provenienti da sensori e dispositivi connessi è diventato fondamentale nelle architetture dei sistemi distribuiti. Questa tesi si colloca in questo scenario e mira a evolvere un'architettura per lo streaming di dati, seguendo l'approccio metodologico e tecnologico della piattaforma iLog.

iLog è stato creato proprio per superare le limitazioni delle pratiche tradizionali di raccolta e analisi dei dati, promuovendo l'integrazione di informazioni numeriche e strutturate con elementi contestuali e umani, spesso trascurati nelle analisi automatizzate. [1]

L'architettura attuale di iLog si basa su Apache Kafka per lo streaming dei dati e utilizza database come Cassandra e PostgreSQL per la loro memorizzazione. Tuttavia, questa soluzione presenta alcuni limiti, come la necessità di interventi manuali per la pulizia dei dati e la difficoltà nell'eseguire query complesse.

Per affrontare queste problematiche, è stata proposta un'evoluzione dell'architettura che prevede l'utilizzo di Delta Lake e MinIO, supportata dal formato Parquet per ottimizzare l'efficienza nella lettura e compressione dei dati. Dopo aver analizzato le possibili fonti di dati, si è deciso di estrarre direttamente le informazioni da Apache Kafka, evitando così le complessità legate all'accesso e alla trasformazione dei dati da database esistenti.

Il sistema proposto sfrutta Apache Flink per il processamento in streaming dei dati e utilizza MinIO come object storage compatibile con Amazon S3 per la loro persistenza. Il formato Parquet è stato scelto per le sue elevate prestazioni nella gestione di dati colonnari, mentre Delta Lake assicura proprietà ACID e funzionalità avanzate di gestione e pulizia.

Durante lo sviluppo, sono emerse alcune criticità riguardanti l'integrazione tra Delta Lake e Apache Flink, che hanno impedito il completamento dell'intera pipeline. Tuttavia, il lavoro svolto ha permesso di delineare possibili direzioni per lo sviluppo futuro, creando così solide basi per il proseguimento del progetto. La tesi si conclude con una riflessione sui limiti riscontrati, sui risultati ottenuti e sulle prospettive di evoluzione del sistema.

1 Introduzione

Recentemente, abbiamo assistito a un'esplosione di dispositivi connessi e sensori intelligenti, il che ha portato a una crescita esponenziale dei dati. Questo scenario ha reso cruciale lo sviluppo di architetture scalabili e flessibili per gestire questa mole di informazioni. In questo contesto, l'analisi dei dati sta diventando sempre più centrale: non si tratta solo di descrivere fenomeni osservabili, ma anche di afferrare le dinamiche sottostanti e supportare decisioni informate. Tuttavia, le tecnologie tradizionali di raccolta e analisi tendono a focalizzarsi solo su dati strutturati e quantitativi, trascurando le dimensioni più profonde e qualitative delle informazioni.

Per affrontare queste sfide, è nato iLog, un framework metodologico e tecnologico progettato per integrare due tipi complementari di dati: da un lato, i Big Data, ovvero dati quantitativi ad alto volume generati in tempo reale da sensori, applicazioni e infrastrutture digitali; dall'altro, i Thick Data, che comprendono informazioni qualitative derivate da esperienze, percezioni e interazioni umane, essenziali per comprendere il contesto e le intenzioni dietro i comportamenti osservati.

[2]

L'architettura iniziale di iLog si basa su Apache Kafka, un sistema di messaggistica distribuito che adotta il modello publish-subscribe. In questo modello, i produttori inviano messaggi su specifici topic, mentre i consumatori si iscrivono a questi topic per ricevere i dati in tempo reale. Kafka offre un'elevata

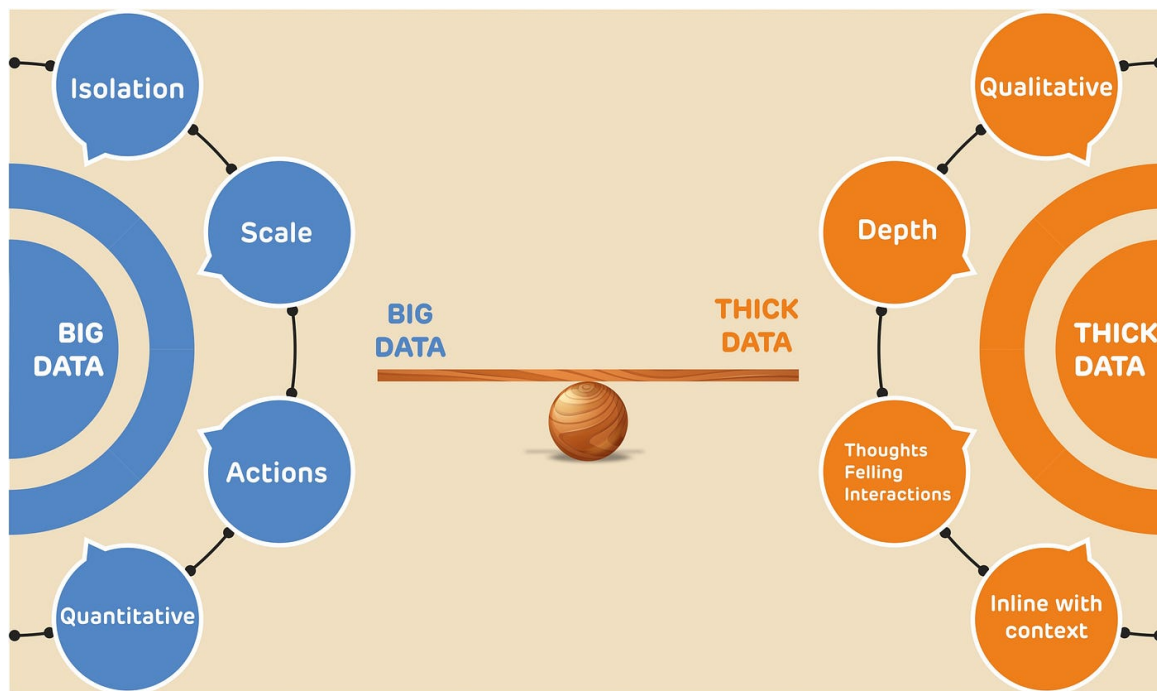


Figura 1.1: Big Data vs Thick Data

affidabilità e scalabilità, rendendolo ideale per scenari di streaming continuo. I dati raccolti tramite Kafka vengono poi memorizzati nei database Cassandra e PostgreSQL, rispettivamente un database NoSQL ad alta disponibilità e uno relazionale, utilizzati per la persistenza e l'interrogazione dei dati.

Tuttavia, questa soluzione presenta alcune criticità. Cassandra, sebbene offra ottime performance in scrittura, richiede una modellazione denormalizzata dei dati e una gestione complessa delle chiavi di partizione, il che può rendere difficili le interrogazioni. Inoltre, la necessità di pulire i dati manualmente rallenta l'efficienza del sistema.

Per affrontare queste sfide, questa tesi propone una nuova architettura che integra tecnologie moderne per la gestione dei dati in streaming e in un formato ottimizzato. Il cuore di questa proposta è l'uso di Delta Lake, un livello di storage open-source costruito sopra Apache Spark, che combina i vantaggi dello storage dei dati immutabili in formato Parquet con funzionalità transazionali ACID, controllo delle versioni, time travel e enforcement dello schema. Delta Lake aiuta a mantenere la qualità e la coerenza dei dati, semplificando anche operazioni di aggiornamento e cancellazione, che sono tradizionalmente complesse nei data lake.

I dati vengono ora estratti direttamente da Kafka e processati in tempo reale tramite Apache Flink, un potente motore di elaborazione per flussi e batch. Flink permette una gestione stateful del flusso e supporta operazioni complesse come windowing, event time processing e aggregazioni in modo scalabile e resistente ai guasti.

Una volta elaborati, i dati vengono salvati in MinIO, un sistema di object storage compatibile con l'API di Amazon S3. MinIO è stato scelto per la sua facilità d'integrazione, l'elevata scalabilità e la capacità di gestire grandi volumi di dati in ambienti containerizzati. Il formato di salvataggio utilizzato è Apache Parquet, un formato colonnare ottimizzato per l'analisi dei dati su larga scala. Parquet migliora notevolmente le prestazioni in lettura e la compressione dei dati rispetto ai formati tradizionali come CSV o JSON.

Il tutto è orchestrato e distribuito su Kubernetes, una piattaforma open-source che automatizza il deployment, la scalabilità e il monitoraggio dei container. Kubernetes consente di gestire in modo

dinamico risorse e servizi, garantendo alta disponibilità e resilienza dell'intero sistema.

2 Stato dell'arte

2.1 Storia delle Tecnologie e dei Metodi per Data Warehouse e Data Lakes

L'evoluzione delle tecnologie per i data warehouse e i data lakes ha subito decenni di cambiamenti significativi, influenzati dall'esplosione dei Big Data, dai progressi nel cloud computing e dalla crescente necessità di analisi più avanzate e in tempo reale. Mentre i data warehouse sono stati a lungo considerati il sistema ideale per archiviare e analizzare dati strutturati provenienti da diverse fonti, i data lakes hanno fornito una soluzione più flessibile per gestire grandi volumi di dati non strutturati e semi-strutturati.

2.1.1 La Nascita dei Data Warehouse

Negli anni '80, con la necessità di centralizzare i dati aziendali per semplificare l'analisi e il reporting, emerse il concetto di data warehouse. I primi sistemi, come quelli creati da Teradata, impiegavano architetture MPP (Massively Parallel Processing) per gestire e analizzare grandi volumi di dati in modo parallelo. Questo approccio fu adottato anche da Oracle e IBM DB2, che iniziarono a supportare sistemi relazionali ottimizzati per operazioni di OLAP (Online Analytical Processing), a differenza dei tradizionali database OLTP (Online Transaction Processing) utilizzati per gestire le operazioni quotidiane. La gestione dei dati in questi sistemi seguiva la metodologia ETL (Extract, Transform, Load), in cui i dati venivano estratti da diverse fonti, trasformati in un formato uniforme e poi caricati nel data warehouse. Questo metodo, che si focalizzava esclusivamente su dati strutturati, era perfetto per attività di reporting e analisi, ma presentava delle limitazioni nel gestire dati non strutturati o semi-strutturati. [3] [4]

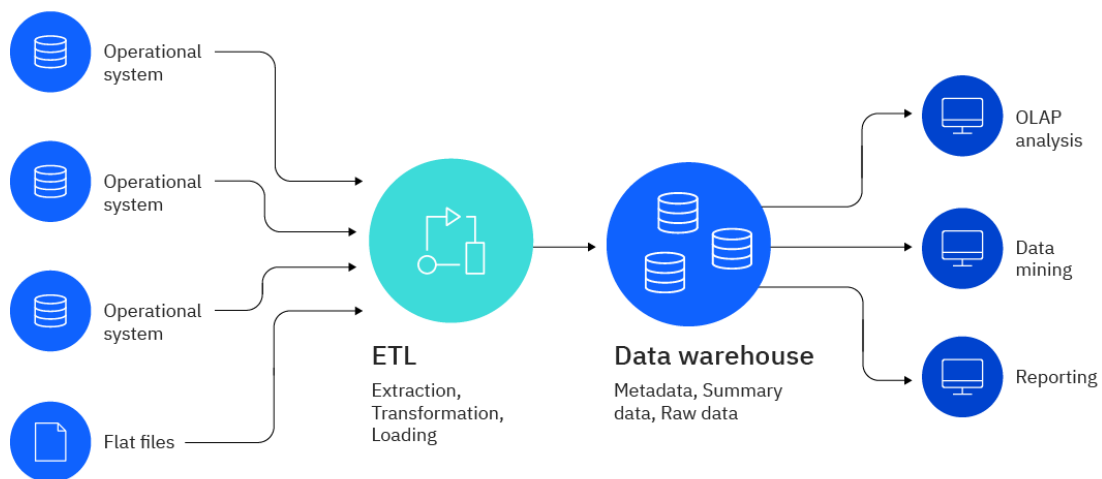


Figura 2.1: Data Warehouse Diagram

2.1.2 L'Avvento dei Data Lakes e di Hadoop

Con l'arrivo di Hadoop negli anni 2000, la gestione dei dati ha subito una trasformazione significativa. Hadoop, che si basa su un framework open-source, ha introdotto il concetto di HDFS (Hadoop

Distributed File System), un sistema di archiviazione distribuita che consente di gestire grandi volumi di dati in modo scalabile e conveniente. Questo approccio ha reso possibile l'archiviazione di dati non strutturati provenienti da varie fonti, come log, social media, sensori IoT e altro ancora. Contemporaneamente, il modello di programmazione MapReduce sviluppato da Hadoop ha reso possibile l'elaborazione parallela dei dati, aumentando l'efficienza nell'analisi di grandi dataset. Mentre i data warehouse continuavano a essere utilizzati per analisi strutturate, i data lakes sono emersi come una soluzione per l'archiviazione e l'elaborazione di dati in un formato più flessibile, senza la necessità di pre-elaborazione. [5] [6]

2.1.3 Il Ruolo del Cloud e la Convergenza tra Data Lakes e Data Warehouse

Con il crescente utilizzo del cloud computing negli anni 2010, tecnologie come Amazon S3 hanno facilitato la scalabilità e l'archiviazione economica dei dati. Apache Spark, lanciato nel 2012, ha rivoluzionato l'elaborazione dei dati, portando benefici significativi sia per i data lakes che per i data warehouse grazie alla sua capacità di elaborare dati in tempo reale, superando le limitazioni di Hadoop in termini di velocità di calcolo. Spark ha permesso alle aziende di elaborare dati provenienti da più fonti e in vari formati in modo molto più efficiente. [7]

Nel 2017, con l'introduzione di Delta Lake, è emersa l'idea del Lakehouse, un'architettura che combina i vantaggi dei data lakes e dei data warehouse. Delta Lake ha introdotto il supporto per transazioni ACID e ha permesso di gestire in modo coerente sia i dati strutturati che non strutturati all'interno dello stesso sistema. Grazie a Delta Lake e alla sua integrazione con Apache Spark, le aziende sono riuscite a superare i limiti dei tradizionali data lakes, che non offrivano garanzie di integrità transazionale, permettendo di applicare tecniche analitiche avanzate ai dati in tempo reale. [8]

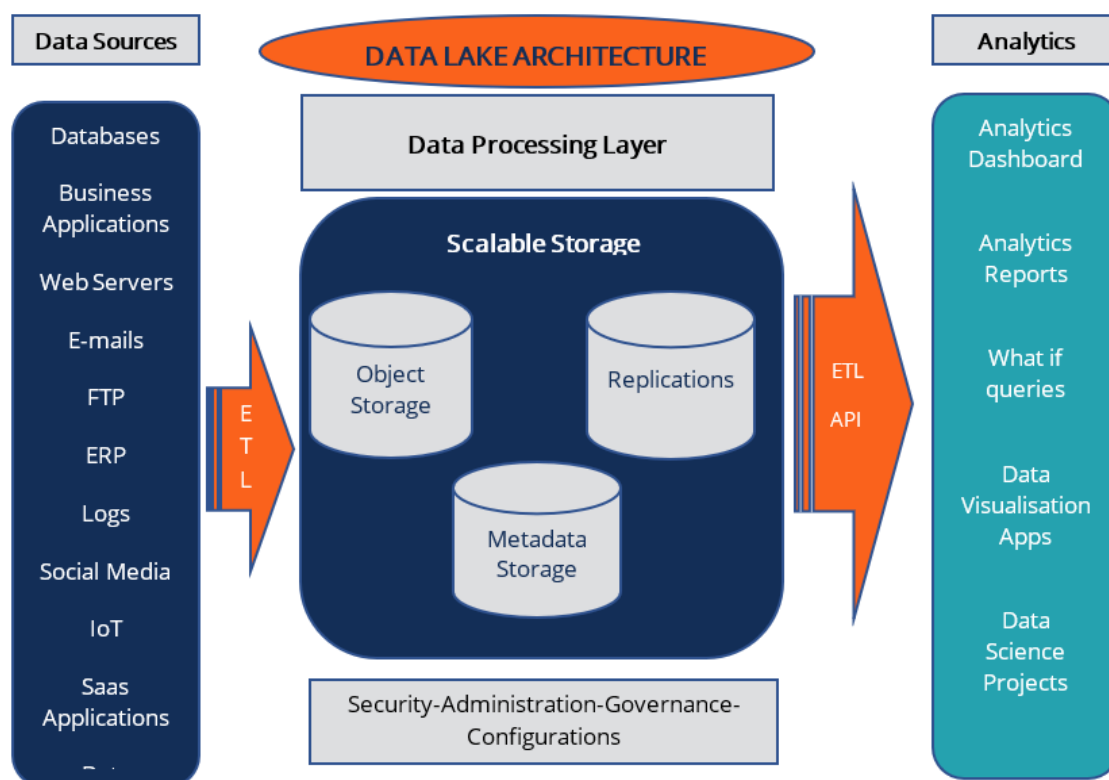


Figura 2.2: Data Lake Diagram

2.1.4 I Data Lakes: Versatilità e Sfide

I data lakes sono stati creati per soddisfare la crescente esigenza di archiviare e analizzare dati non strutturati. Grazie alla loro flessibilità, i data lakes consentono alle organizzazioni di raccogliere qualsiasi tipo di dato senza la necessità di strutturarli in anticipo. Questo approccio ha portato a notevoli

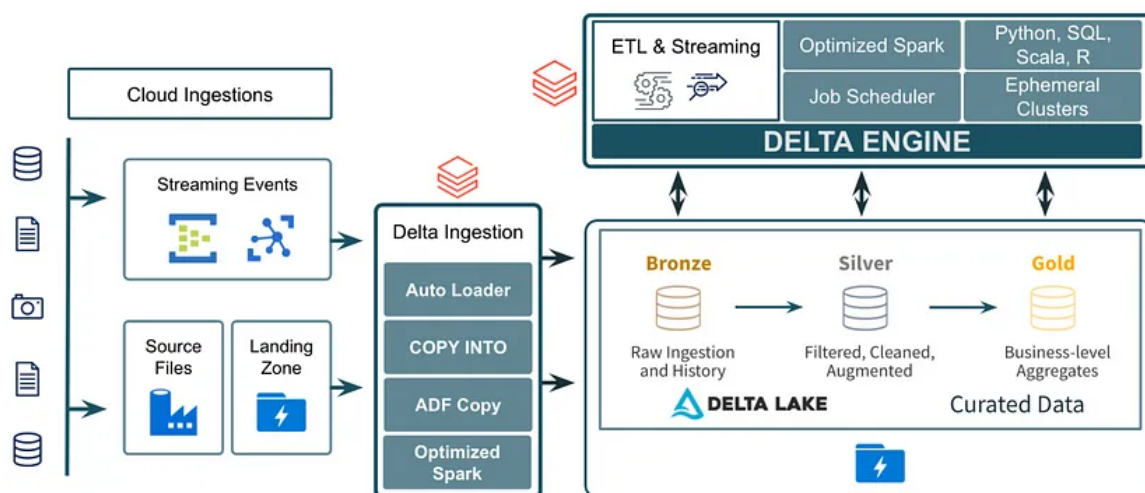


Figura 2.3: Delta Lake Diagram

vantaggi in termini di capacità di archiviazione, ma ha anche sollevato nuove sfide riguardanti la qualità e la governance dei dati. L'assenza di strutture predefinite può causare problemi di gestione, difficoltà nell'estrazione di valore analitico e complicazioni nella sicurezza dei dati. [9]

L'emergere di tecnologie come Apache Kafka e Apache Flink ha ulteriormente trasformato il concetto di data lakes, rendendo possibile la gestione e l'elaborazione dei dati in tempo reale. Apache Kafka ha introdotto il concetto di streaming dei dati, consentendo la raccolta e l'elaborazione continua dei dati, mentre Apache Flink ha migliorato la capacità di eseguire elaborazioni in tempo reale sui flussi di dati. [10] [11]

2.1.5 La Nuova Era: Integrazione tra Data Lakes e Data Warehouse

Nel contesto attuale, l'architettura moderna dei dati si sta sempre più orientando verso la fusione tra data lakes e data warehouse. Il concetto di Lakehouse è emerso per combinare i vantaggi di entrambi i modelli, consentendo alle aziende di gestire sia dati strutturati che non strutturati, garantendo al contempo coerenza e governance sui dati. Grazie a Delta Lake, le aziende possono ora beneficiare di transazioni ACID, supporto per la versione dei dati e miglioramenti nelle prestazioni analitiche.

2.1.6 Metodologie e Nuove Pratiche

Con l'evoluzione della tecnologia, anche i metodi di gestione dei dati hanno subito cambiamenti notevoli. L'approccio tradizionale ETL (Extract, Transform, Load) è stato progressivamente sostituito dal modello ELT (Extract, Load, Transform), in cui i dati vengono caricati direttamente nel data lake e poi trasformati per l'analisi. Questo nuovo metodo ha guadagnato popolarità grazie alla maggiore potenza di calcolo disponibile e alla flessibilità offerta dai data lakes.

Inoltre, l'emergere di strumenti per la governance dei dati come Apache Ranger e Delta Lake ha consentito un controllo più preciso sulla qualità e sulla sicurezza dei dati, affrontando così una delle principali sfide nella gestione dei data lakes.

2.1.7 Conclusioni

L'evoluzione dei data warehouse e dei data lakes ha reso possibile affrontare le sfide nella gestione e nell'analisi dei big data, portando alla nascita di nuove architetture come il Lakehouse. Le soluzioni moderne combinano le capacità di archiviazione dei data lakes con quelle di elaborazione dei data warehouse, permettendo alle aziende di gestire in modo efficace sia i dati strutturati che quelli non strutturati, e di utilizzare l'analisi in tempo reale per rispondere alle esigenze sempre più complesse del business.

2.2 Architettura Lambda e Delta

L'enorme crescita dei dati generati in tempo reale da sensori, dispositivi IoT e sistemi digitali ha reso necessaria la progettazione di architetture in grado di processare e analizzare dati sia in streaming che in batch. In questo contesto si inserisce l'evoluzione delle cosiddette *architetture Lambda e Delta*, due paradigmi progettuali fondamentali nel moderno panorama dei sistemi di data processing distribuito.

2.2.1 Architettura Lambda

L'architettura Lambda è stata introdotta da Nathan Marz nel 2011 nel contesto di sistemi su larga scala come quelli sviluppati in Twitter, per affrontare le limitazioni dei sistemi di sola analisi batch. Essa si basa su un modello ibrido che distingue due livelli principali:

- **Batch layer:** gestisce l'intero dataset in modo immutabile e ricostruisce periodicamente una vista aggregata dei dati. Generalmente utilizza tecnologie come Apache Hadoop o Apache Spark.
- **Speed layer:** gestisce il flusso di dati in tempo reale per fornire aggiornamenti rapidi, sacrificando parzialmente la precisione a favore della velocità. Viene solitamente implementato con Apache Storm o Apache Flink.
- **Serving layer:** espone i risultati finali all'utente o ai sistemi di consumo, integrando i risultati dei due layer precedenti.

L'obiettivo è fornire una visione accurata e aggiornata dei dati combinando l'accuratezza del batch con la velocità dello streaming. Tuttavia, questa architettura presenta alcune criticità:

- Duplicazione della logica di business tra batch e speed layer, che rende difficile la manutenzione del codice;
- Complessità operativa, dovuta alla gestione parallela di due sistemi eterogenei;
- Latenza del batch layer, che può richiedere tempi lunghi per la ricostruzione dei risultati.

[12]

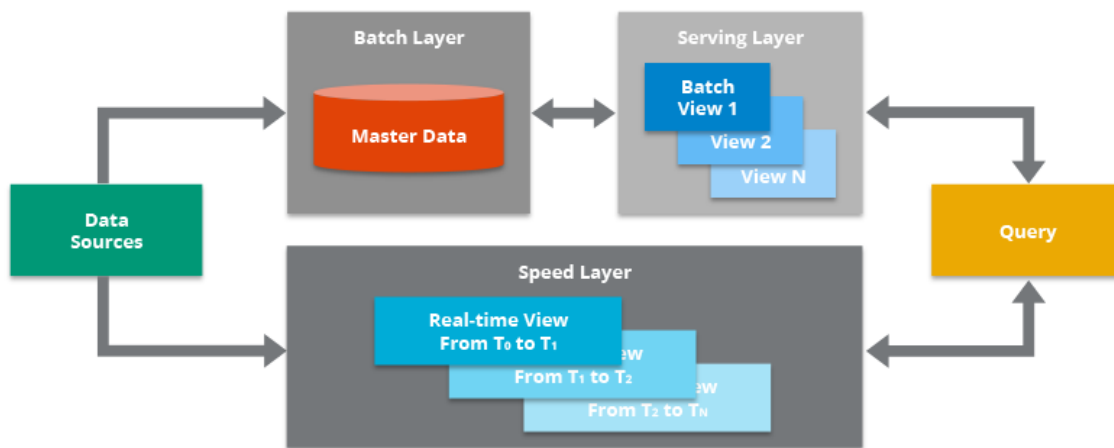


Figura 2.4: Lambda Architecture

2.2.2 Architettura Delta

Per affrontare questi limiti, è stata introdotta l'*architettura Delta*, resa popolare da Databricks insieme al motore Delta Lake. A differenza dell'approccio Lambda, la Delta Architecture propone una semplificazione del modello, in cui batch e streaming sono unificati in un unico livello di elaborazione, grazie all'evoluzione di tecnologie come Apache Spark Structured Streaming.

Questa architettura si fonda su:

- **Un'unica sorgente di verità dei dati:** i dati grezzi vengono salvati in formato append-only, spesso in Parquet;
- **Streaming ingestion:** i dati vengono ingestati in tempo reale attraverso tecnologie come Kafka;
- **Delta Lake:** introduce una gestione transazionale (ACID), versionamento, e schema enforcement direttamente su file system distribuiti;
- **Processing layer unificato:** batch e streaming utilizzano la stessa logica di elaborazione, semplificando la gestione del codice.

L'architettura Delta consente quindi una semplificazione significativa rispetto al modello Lambda, mantenendo comunque basse latenze e elevata affidabilità. Inoltre, la compatibilità con strumenti come Apache Spark, Databricks, Delta Live Tables e l'integrazione con sistemi di storage economici e scalabili (ad esempio, Amazon S3 o MinIO) rendono questa soluzione particolarmente adatta a contesti cloud-native e ad ambienti containerizzati. [13]

2.2.3 Verso una Unified Architecture

Con la crescente maturità degli strumenti di *streaming stateful*, come Apache Flink, e l'evoluzione dei data lake verso forme più intelligenti e transazionali (i cosiddetti *lakehouse*), si sta affermando un nuovo paradigma architetturale: quello della *Unified Architecture*. Questo approccio prevede che un unico motore sia in grado di gestire sia stream che batch, su una sola pipeline logica, evitando duplicazioni e garantendo consistenza.

Flink, ad esempio, grazie al suo supporto per *streaming SQL*, *event time processing* e *checkpointing fault-tolerant*, si configura come una tecnologia chiave per architetture unificate. Tuttavia, la piena integrazione con formati transazionali come Delta Lake è ancora un tema aperto, in particolare per quanto riguarda il supporto in modalità streaming. [14]

2.2.4 Conclusioni

L'evoluzione delle architetture Lambda verso modelli Delta e Unified evidenzia il continuo sforzo nella comunità scientifica e industriale per ridurre la complessità, aumentare l'efficienza e migliorare l'accessibilità dei dati. In tale prospettiva, la proposta descritta in questa tesi si inserisce in linea con questi principi: un sistema moderno, scalabile e semplificato, che sfrutta le potenzialità di strumenti avanzati come Delta Lake, Apache Flink e MinIO per realizzare una pipeline efficiente di gestione dati in tempo reale.

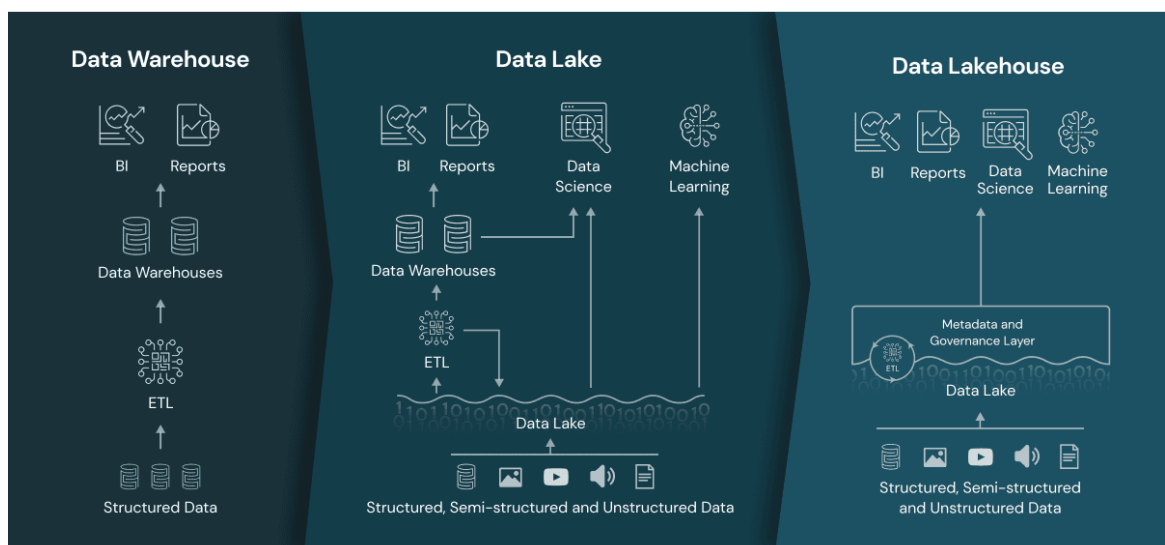


Figura 2.5: Delta vs Lambda vs Lakehouse

2.3 Dallo Stato dell'Arte alla Soluzione: Delta Lake

L'evoluzione delle tecnologie per la gestione dei dati ha reso necessaria una soluzione che unisse la flessibilità dei data lakes con la solidità dei data warehouse, superando le limitazioni esistenti, come la gestione dei dati non strutturati, l'assenza di garanzie di coerenza e le difficoltà nel fare aggiornamenti e cancellazioni sui dati.

Mentre i data warehouse tradizionali, basati su sistemi relazionali, garantivano transazioni ACID (Atomicità, Consistenza, Isolamento, Durabilità), i data lakes non riuscivano a mantenere lo stesso livello di coerenza. Sebbene i data lakes offrano una soluzione economica e scalabile per gestire grandi volumi di dati, presentano limitazioni dovute alla mancanza di meccanismi di gestione transazionale e alla difficoltà di eseguire operazioni di aggiornamento o cancellazione su dati già scritti.

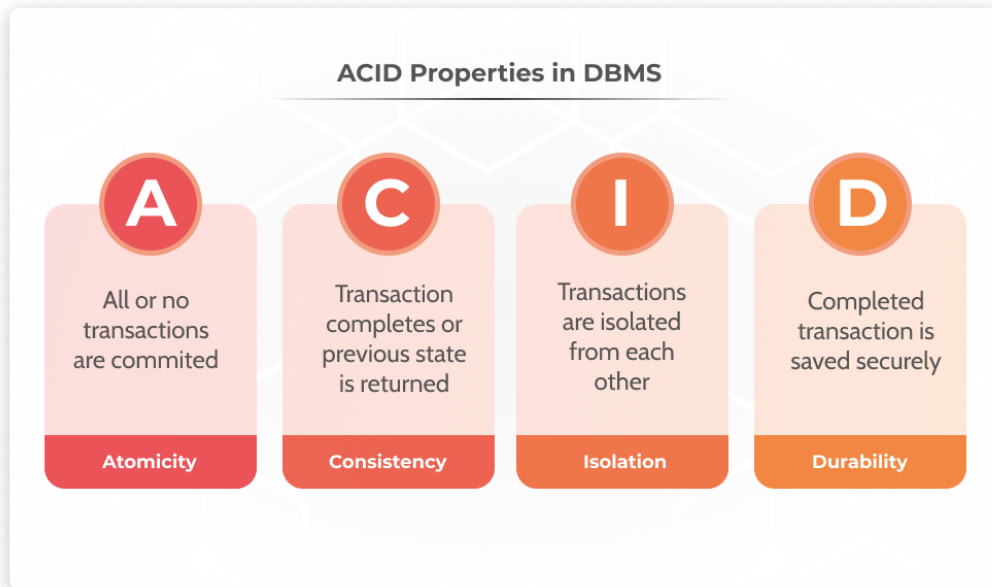


Figura 2.6: ACID Transaction

Delta Lake è stato sviluppato proprio per colmare questo divario. Combinando le caratteristiche dei data lakes con le garanzie dei data warehouse, Delta Lake offre una soluzione integrata che supporta transazioni ACID, la gestione delle versioni dei dati e operazioni di aggiornamento e cancellazione in modo efficiente.

2.3.1 Le Funzionalità Chiave di Delta Lake

Delta Lake presenta diversi vantaggi rispetto ai tradizionali data lakes. Tra le sue caratteristiche principali, possiamo evidenziare:

- **Transazioni ACID:** Delta Lake offre garanzie ACID per i dati, consentendo operazioni sicure di scrittura, aggiornamento e cancellazione, anche in ambienti distribuiti. Questo aiuta a prevenire i problemi di inconsistenza dei dati che possono sorgere nei tradizionali data lakes.
- **Versioning e Time Travel:** Con il versioning dei dati, Delta Lake permette l'accesso ai dati storici attraverso il concetto di *time travel*. Questo consente di recuperare dati a un determinato punto nel tempo, facilitando l'analisi retrospettiva e il ripristino da errori o modifiche indesiderate.
- **Eliminazione e aggiornamenti efficienti:** A differenza dei tradizionali file Parquet, che non supportano facilmente operazioni di aggiornamento o cancellazione, Delta Lake permette di eseguire operazioni MERGE, UPDATE e DELETE sui dati in modo ottimizzato, senza dover riscrivere l'intero dataset.
- **Schema enforcement e schema evolution:** Delta Lake impone il controllo dello schema (schema enforcement) per garantire che i dati scritti seguano una struttura compatibile con il

formato predefinito. Inoltre, supporta l'evoluzione dello schema (schema evolution), consentendo modifiche al formato dei dati nel tempo, senza compromettere la coerenza dei dati esistenti.

2.3.2 Come Funziona Delta Lake

Delta Lake è progettato come uno strato di archiviazione su Apache Spark, dove i dati vengono salvati come file Parquet nel sistema di file sottostante, come HDFS o S3. La principale differenza rispetto ai sistemi di archiviazione tradizionali è l'aggiunta di un log transazionale, il Delta Log, che registra tutte le modifiche ai dati, comprese le operazioni di scrittura, aggiornamento e cancellazione. Questo log è fondamentale per garantire la coerenza e le transazioni ACID, anche in ambienti distribuiti e su larga scala.

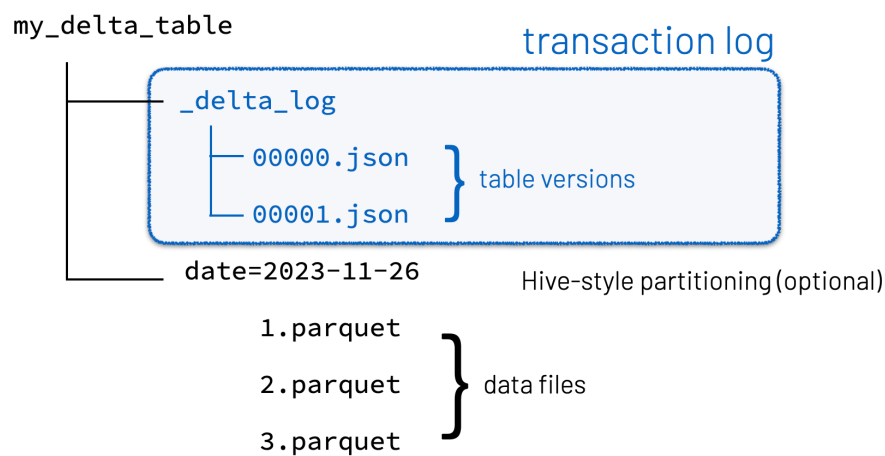


Figura 2.7: esempio Delta Log

Ogni volta che un processo di scrittura modifica i dati, il Delta Log viene aggiornato per riflettere tali cambiamenti. Quando i dati vengono letti, il sistema consulta il Delta Log per identificare quali file devono essere letti e per applicare in modo coerente tutte le modifiche. In questo modo, Delta Lake riesce a mantenere l'integrità dei dati, anche durante operazioni complesse di aggiornamento o cancellazione.

Un'altra caratteristica notevole di Delta Lake è il time travel, che permette di accedere a versioni precedenti dei dati. Grazie al Delta Log, è possibile "viaggiare nel tempo" e visualizzare i dati a un determinato punto temporale, come se si stesse osservando una versione passata del dataset. Questa funzionalità è particolarmente utile per operazioni di auditing, recupero da errori o semplicemente per condurre analisi storiche.

2.3.3 La Soluzione Completa: Delta Lake come Piattaforma di Big Data

Delta Lake è una soluzione completa per la gestione e l'elaborazione dei big data, combinando la potenza dei data lakes con le garanzie di consistenza tipiche dei data warehouse. Le sue funzionalità avanzate, come le transazioni ACID, il versioning dei dati e il supporto per operazioni in tempo reale, lo rendono particolarmente adatto a contesti complessi e dinamici, dove i dati devono essere costantemente aggiornati e analizzati in modo efficiente. La sua capacità di garantire consistenza e affidabilità durante l'elaborazione dei dati, semplificando la complessità dei sistemi di big data, lo rende una scelta ideale per le organizzazioni che devono gestire grandi volumi di dati sia strutturati che non strutturati in

tempo reale.

3 Architettura Corrente

3.1 Diagramma sistema attuale

L'architettura del sistema usato correntemente è la seguente:

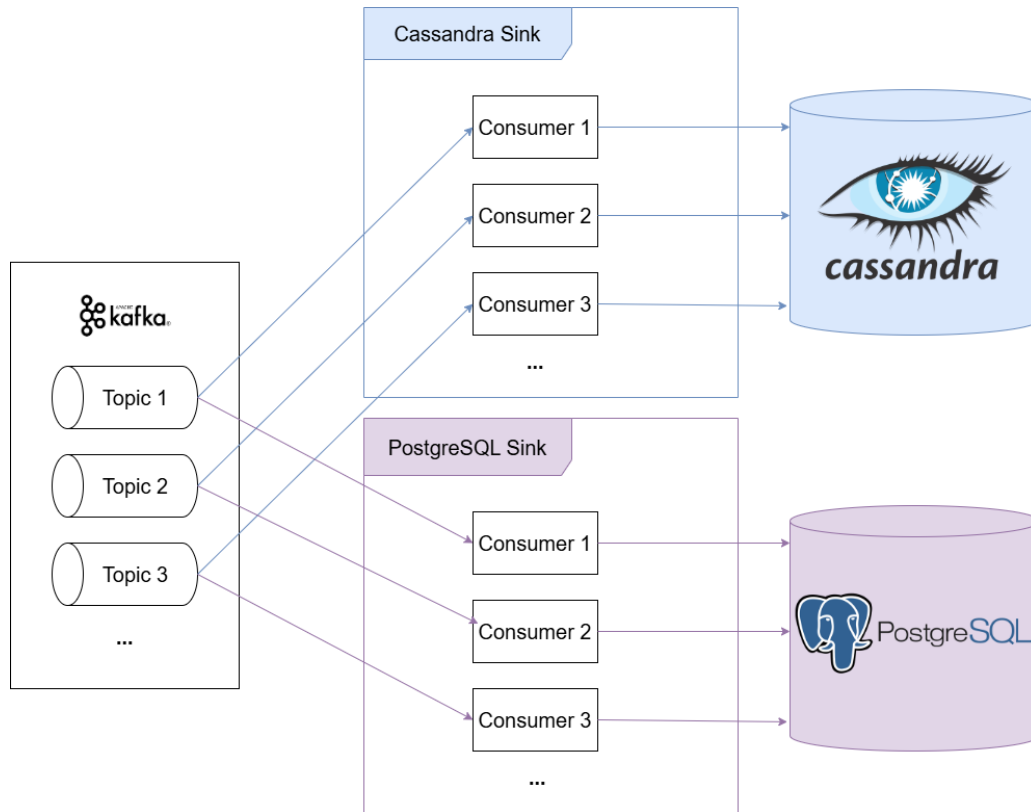


Figura 3.1: Current Architecture

L'architettura che abbiamo progettato si fonda sull'uso di Apache Kafka come sistema di messaggistica centrale. Qui, i producer di Kafka si occupano di raccogliere dati da varie fonti e di inserirli in specifici stream, organizzati per argomenti tematici. Ogni topic funge da canale logico dove si accumulano i messaggi relativi a un certo tipo di dato o evento. Una volta che i dati vengono pubblicati su Kafka, vengono consumati in parallelo da due diversi sink: uno per Cassandra e l'altro per PostgreSQL. Il sink di Cassandra legge i messaggi dallo stream e li archivia nel database Cassandra, che è ottimizzato per scritture rapide e per gestire grandi volumi di dati distribuiti. Nel frattempo, il sink di PostgreSQL svolge un compito simile, consumando gli stessi dati dallo stream e scrivendoli nel database PostgreSQL, perfetto per la gestione relazionale e per eseguire query SQL complesse. Questo meccanismo di doppio consumo permette di sfruttare i punti di forza di entrambi i database, mantenendo una replica coerente dei dati. L'approccio assicura scalabilità e flessibilità, consentendo l'elaborazione in tempo reale e una persistenza differenziata in base alle esigenze analitiche o applicative. In questo modo, Kafka funge da ponte centrale e buffer temporaneo, garantendo affidabilità nella trasmissione dei dati verso entrambi i sistemi di destinazione.

Inoltre, si è scelto di continuare a usare i seguenti strumenti:

3.1.1 Kubernetes

Lo sviluppo dell'applicazione è stato fatto usando Kubernetes. Esso è una piattaforma open-source per l'automazione del deployment, della gestione e della scalabilità di applicazioni containerizzate.

Kubernetes è comunemente utilizzato in ambienti di produzione per applicazioni di microservizi, dove ogni microservizio è containerizzato e può essere gestito autonomamente. Viene spesso combinato con tecnologie come Docker per la containerizzazione e con Helm per la gestione dei pacchetti. Kubernetes offre numerosi vantaggi che lo rendono una scelta ideale per la gestione di applicazioni distribuite. La portabilità è uno dei suoi principali punti di forza, poiché essendo agnostico alla piattaforma, consente alle applicazioni di essere eseguite su qualsiasi ambiente, sia esso cloud o on-premises. Inoltre, Kubernetes facilita l'automazione di compiti ripetitivi come la distribuzione, il bilanciamento del carico, il monitoraggio e il failover, riducendo il carico operativo e aumentando l'efficienza. La scalabilità e la flessibilità sono altre caratteristiche chiave, poiché Kubernetes permette di adattare automaticamente le risorse in base al traffico o alle necessità delle applicazioni, ottimizzando l'uso delle risorse. Infine, grazie alla sua capacità di garantire alta disponibilità e resilienza, Kubernetes assicura che le applicazioni rimangano operative anche in caso di guasti hardware o software, mantenendo il sistema stabile e affidabile. [15]

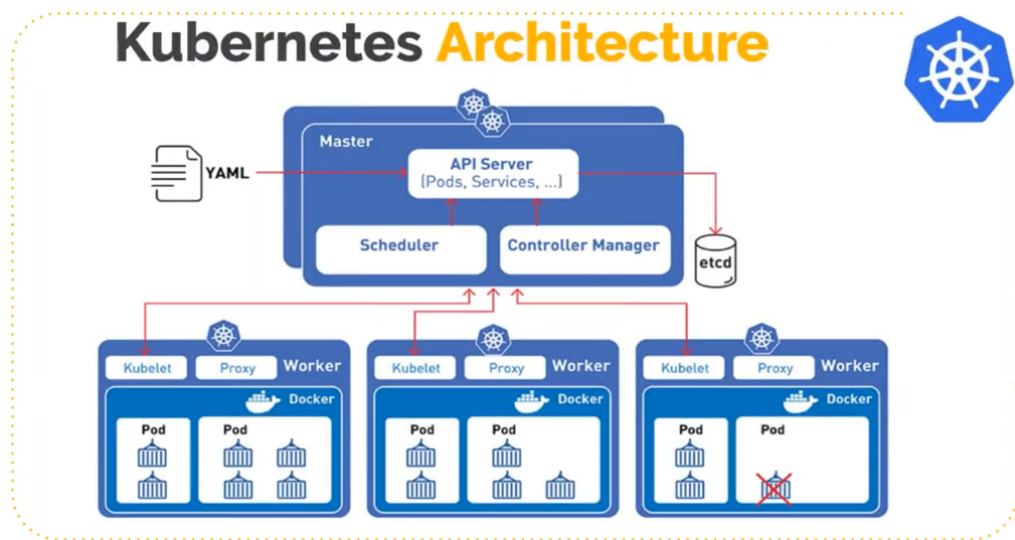


Figura 3.2: Kubernetes Architecture

3.1.2 Apache Kafka

Apache Kafka è un sistema distribuito composto da server e client, che comunicano tramite un protocollo TCP ad alte prestazioni. Esso può essere implementato su hardware fisico, macchine virtuali e container; e sia in ambienti on-premise che nel cloud. Concettualmente si possono identificare le seguenti componenti:

- **Server:** Kafka viene eseguito come un cluster di uno o più server, che possono essere distribuiti su più data center o regioni del cloud. Alcuni di questi server svolgono la funzione di broker, gestendo lo storage dei dati. Grazie alla sua architettura scalabile e tollerante ai guasti, un cluster Kafka è progettato per garantire operazioni senza interruzioni e senza perdita di dati anche in caso di danni: se un server si guasta, gli altri server prendono automaticamente il suo posto.
- **Client:** I client di Kafka consentono di sviluppare applicazioni distribuite e microservizi che leggono, scrivono e processano flussi di eventi in parallelo e su larga scala, anche in caso di problemi di rete o guasti hardware.
- **Eventi:** Quando si legge o scrive dati su Kafka, lo si fa sotto forma di eventi. Concettualmente, un evento ha una chiave, un valore, un timestamp e degli header di metadati opzionali.
- **Producer e Consumer:** I producer sono le applicazioni client che pubblicano (scrivono) eventi su Kafka, mentre i consumer sono quelle che si sottoscrivono (leggono e processano) a questi eventi. In Kafka, i producer e i consumer sono completamente separati e indipendenti l'uno dall'altro; ed esso è un aspetto fondamentale per garantire l'alta scalabilità che distingue Kafka.

- **Topic:** Gli eventi sono organizzati e memorizzati in modo durevole all'interno di topic. In modo semplificato, un topic può essere paragonato a una cartella di un file system, mentre gli eventi corrispondono ai file contenuti al suo interno. I topic in Kafka sono sempre multi-producer e multi-subscriber: un topic può avere zero, uno o più producer che scrivono eventi al suo interno, così come zero, uno o più consumer che si sottoscrivono per leggere questi eventi. Gli eventi di un topic possono essere letti tutte le volte che è necessario: a differenza dei tradizionali sistemi di messaggistica, gli eventi non vengono eliminati dopo essere stati consumati. Invece, si può configurare per quanto tempo Kafka deve conservare gli eventi, dopo di che gli eventi più vecchi verranno scartati. Le performance di Kafka rimangono praticamente costanti indipendentemente dalla dimensione dei dati, quindi conservare i dati per lunghi periodi è perfettamente fattibile.
- **Partizioni:** I topic sono suddivisi in partizioni, il che significa che un topic è distribuito su più "container" situati su diversi broker Kafka. Questa distribuzione dei dati è cruciale per la scalabilità, in quanto consente alle applicazioni client di leggere e scrivere dati contemporaneamente su più broker. Quando un nuovo evento viene pubblicato in un topic, esso viene effettivamente aggiunto a una delle partizioni del topic. Gli eventi con la stessa chiave vengono scritti nella stessa partizione, e Kafka garantisce che ogni consumer di una determinata partizione legga sempre gli eventi in quell'ordine, esattamente come sono stati scritti.

[11]



Figura 3.3: Kafka Diagram

4 Requisiti

Lo sviluppo del sistema descritto in questa tesi nasce dall'esigenza di creare una soluzione affidabile per il trasferimento continuo di dati da una piattaforma di raccolta, come Apache Kafka, a uno storage finale, come MinIO, utilizzando il formato Parquet. Questo processo deve avvenire in modalità streaming, garantendo alti livelli di integrità, scalabilità e compatibilità con le tecnologie moderne. In questo capitolo, verranno illustrati i requisiti del sistema, suddivisi in requisiti funzionali, che definiscono le operazioni che il sistema deve svolgere, e requisiti non funzionali, che specificano le qualità attese.

4.1 Requisiti Funzionali

- **RF1 – Ingestione dati da Apache Kafka:** Il sistema deve essere in grado di connettersi a uno o più topic di Apache Kafka e leggere i dati pubblicati in tempo reale. Questa operazione rappresenta il punto di ingresso principale del flusso di dati e deve essere continua, scalabile e resiliente. Il connettore deve gestire i commit degli offset per garantire un'elaborazione esatta (exactly-once) o almeno una volta (at-least-once), a seconda delle necessità di consistenza.
- **RF2 – Scrittura dei dati in formato Parquet:** Dopo la fase di elaborazione, i dati devono essere convertiti e serializzati in formato Parquet, un formato colonnare che offre efficienza nella compressione e nell'analisi. Il sistema deve gestire la partizionatura dei file e la scrittura atomica per garantire coerenza ed efficienza nelle operazioni di lettura e analisi successive.

- **RF3 – Archiviazione dei dati su MinIO:** I file Parquet generati devono essere salvati su MinIO, un object storage che supporta l'interfaccia S3. Il sistema deve gestire la connessione allo storage, organizzare le directory (ad esempio, per data, tipo di dato o altre chiavi logiche) e inviare i file in modo sicuro e idempotente.
- **RF4 – Elaborazione in tempo reale:** Il sistema deve elaborare i dati con la minima latenza possibile, evitando la necessità di batch o operazioni asincrone troppo lente. L'elaborazione in tempo reale è cruciale per garantire che i dati siano disponibili nel minor tempo possibile per analisi o archiviazione.
- **RF5 – Gestione dei fallimenti:** Il sistema deve includere meccanismi robusti per la gestione degli errori. In caso di interruzioni di rete, errori nei dati o problemi di scrittura, deve essere in grado di riprendere l'elaborazione senza perdere dati, implementando strategie di retry o registrando gli eventi problematici, per garantire tracciabilità e possibilità di intervento.

4.2 Requisiti Non Funzionali

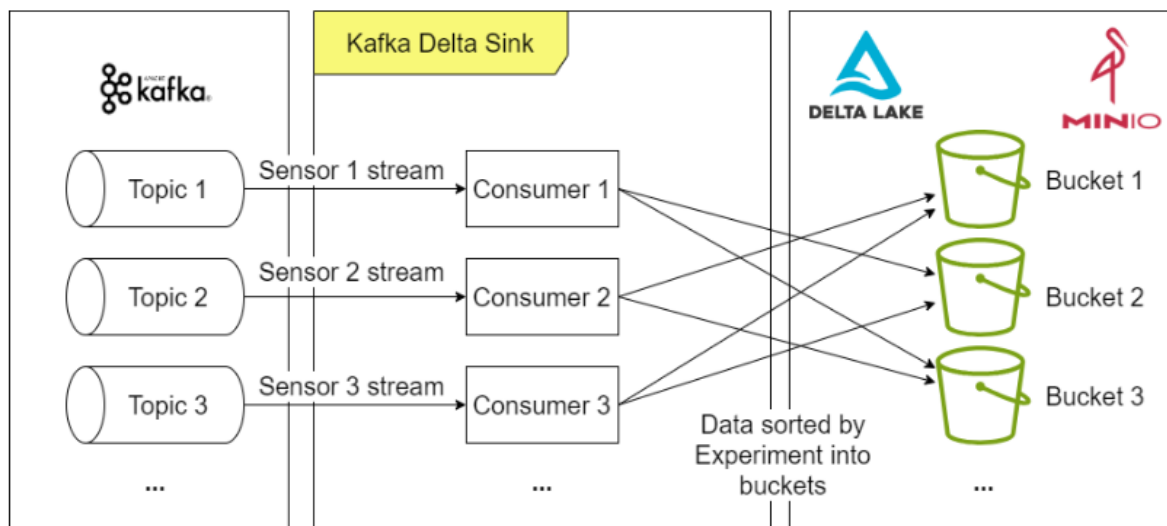
- **RNF1 – Scalabilità:** Il sistema deve essere capace di scalare orizzontalmente, il che significa che deve poter aumentare le sue capacità di ingestione ed elaborazione distribuendo il carico su più istanze. Questo è particolarmente cruciale in situazioni con un alto volume di dati e può essere realizzato sfruttando l'architettura distribuita di Apache Kafka, Apache Flink e MinIO.
- **RNF2 – Performance:** Il tempo che intercorre tra la ricezione di un dato in Kafka e la sua disponibilità nello storage finale deve essere ridotto al minimo. Il sistema deve supportare throughput elevati (ad esempio, migliaia di messaggi al secondo) mantenendo la latenza sotto una soglia definita, anche quando i carichi aumentano.
- **RNF3 – Affidabilità:** La soluzione deve garantire che nessun dato venga perso o duplicato. Per questo, è fondamentale implementare tecniche di gestione delle transazioni, checkpointing e un attento controllo dell'elaborazione degli offset Kafka, oltre alla scrittura su Delta Lake.
- **RNF4 – Modularità e manutenibilità:** Ogni componente del sistema (ingestione, elaborazione, storage) deve essere sviluppato come un modulo indipendente, facilmente aggiornabile o sostituibile. Questo approccio rende la manutenzione più semplice, facilita l'aggiunta di nuove funzionalità e permette di adattare il sistema a nuove esigenze.
- **RNF5 – Interoperabilità:** L'uso di formati standard come Parquet e protocolli aperti (Kafka protocol, S3 API) consente di integrare il sistema con altri strumenti di analisi, visualizzazione o machine learning, garantendo così flessibilità e possibilità di estensione.
- **RNF6 – Monitoraggio e osservabilità:** Il sistema deve avere a disposizione strumenti di osservabilità, come metriche, log e alerting, per tenere sotto controllo lo stato delle componenti, il volume dei dati elaborati, le latenze, gli errori e i possibili colli di bottiglia. Queste informazioni sono essenziali per una gestione operativa efficace e per diagnosticare eventuali problemi.

La definizione accurata dei requisiti ha guidato le scelte architetturali e tecnologiche descritte nei capitoli successivi. Essa costituisce anche la base per la valutazione dell'efficacia della soluzione proposta e l'individuazione delle sue potenzialità evolutive.

5 Soluzione proposta

5.1 Architettura e flusso dei dati

Lo schema dell'architettura del sistema è la seguente:



In breve, dati di iLog generati dai sensori vengono inviati a Kafka, dove ogni sensore ha il proprio stream dedicato, organizzato in topic specifici. Dal lato del Sink (così chiamato il sistema), dei consumer dedicati si occupano di leggere i dati da ciascun topic, assicurando un'elaborazione efficiente e scalabile. Una volta acquisiti, i dati vengono ulteriormente organizzati e suddivisi in base all'Esperimento iLog di riferimento. Questa suddivisione avviene all'interno di bucket predefiniti in MinIO, dove ogni Esperimento ha un proprio spazio dedicato per l'archiviazione. La modularità è implicita nel design dell'architettura a componenti. **(RF1 – Ingestione dati da Apache Kafka, RNF1 – Scalabilità, RNF4 – Modularità e manutenibilità)**

5.2 Tecnologie aggiunte

5.2.1 Delta Lake

Delta Lake è una storage layer open-source progettata per garantire transazioni ACID (Atomicità, Consistenza, Isolamento e Durabilità) su Apache Spark e Big Data. È stata sviluppata da Databricks e si basa su un formato di file Parquet, arricchendolo con funzionalità avanzate come il versioning dei dati, la gestione delle transazioni e il supporto per le operazioni di streaming e batch processing. La sua funzione principale è quella di fornire una gestione robusta dei dati in ambienti di big data analytics, garantendo la consistenza e l'affidabilità durante l'elaborazione dei dati, riducendo la complessità tipica dei sistemi di big data.

- **Transazioni ACID:** Delta Lake offre garanzie ACID per i dati, permettendo di eseguire operazioni di scrittura, aggiornamento e cancellazione in modo sicuro, evitando problemi di inconsistenza nei dati anche in ambienti distribuiti.
- **Versioning e Time Travel:** Delta Lake mantiene una cronologia delle versioni dei dati, consentendo di accedere ai dati a uno stato precedente attraverso il concetto di time travel. Questo permette di recuperare versioni specifiche dei dati, utile per analisi retrospettive o per recuperare dati in caso di errori.
- **Eliminazione e aggiornamenti efficienti:** Delta Lake supporta operazioni MERGE, UPDATE e DELETE sui dati. Queste operazioni sono ottimizzate rispetto alla tradizionale manipolazione di dati in Parquet, che non consente di aggiornare facilmente i dati. In Delta Lake, gli aggiornamenti vengono eseguiti in modo efficiente, senza la necessità di riscrivere l'intero dataset.
- **Schema enforcement e schema evolution:** Delta Lake impone una verifica dello schema (schema enforcement), che garantisce che i dati che vengono scritti abbiano una struttura compatibile con il formato predefinito. Inoltre, supporta l'evoluzione dello schema (schema evolution), consentendo modifiche al formato dei dati nel tempo, senza compromettere la consistenza dei dati esistenti.

Riguardo al suo funzionamento, Delta Lake è implementato su Apache Spark come una storage layer, dove i dati scritti vengono memorizzati come file Parquet nel sistema di file sottostante (come HDFS, S3, ecc.), ma con l'aggiunta di un log transazionale per ogni operazione di scrittura, che garantisce la coerenza e le transazioni ACID. Il log, denominato Delta Log, tiene traccia di tutte le modifiche ai dati, comprese le operazioni di aggiornamento, cancellazione e aggiunta. Quando un processo legge i dati, consulta il Delta Log per determinare quali file devono essere letti e applica tutte le modifiche in modo coerente. Inoltre, Delta Lake supporta il time travel, che consente di accedere ai dati storici consultando la cronologia dei cambiamenti registrata nel Delta Log, permettendo di leggere i dati a uno specifico punto temporale come se si stesse visualizzando un'istantanea di quel momento. [8] (**RF5 – Gestione dei fallimenti, RNF3 – Affidabilità**)

5.2.2 Apache Flink

Apache Flink è un framework open-source per l'elaborazione in streaming e batch di grandi volumi di dati in tempo reale. È progettato per gestire flussi di dati in modo altamente scalabile, distribuito e con basse latenze, e viene utilizzato principalmente in scenari di Big Data e Data Engineering. Flink è particolarmente adatto per applicazioni che richiedono elaborazioni complesse su flussi di dati continui, come monitoraggio in tempo reale, analisi di eventi, e gestione delle informazioni provenienti da sensori o dispositivi IoT. I vantaggi di Flink sono i seguenti:

- **Elaborazione in tempo reale:** Flink è costruito per l'elaborazione di flussi di dati in tempo reale, con un basso ritardo nelle operazioni di elaborazione e una gestione ottimizzata dei dati in streaming. Può essere utilizzato per analisi in tempo reale su grandi volumi di dati provenienti da diverse fonti.
- **Scalabilità e distribuzione:** Flink è progettato per essere altamente scalabile, permettendo di gestire enormi volumi di dati distribuiti su cluster di macchine. Grazie alla sua architettura distribuita, può essere facilmente scalato orizzontalmente, adattandosi alle esigenze del carico di lavoro.
- **Elaborazione di dati sia in batch che in streaming:** Flink è unico nel suo supporto per elaborazione unificata sia in tempo reale (streaming) che in batch. Questo permette di eseguire la stessa pipeline di dati in modo continuo (streaming) o in modo batch (per l'analisi di grandi dataset preesistenti).
- **Flessibilità nell'uso:** Flink supporta una varietà di sink e source per l'integrazione con diversi sistemi di archiviazione, database e piattaforme di messaggistica, come Kafka, HDFS, Cassandra e molti altri. Inoltre, offre un linguaggio di query basato su SQL per semplificare le operazioni di elaborazione dei dati.
- **Elaborazione stateful:** Flink è in grado di gestire e calcolare stati complessi, garantendo che anche in caso di guasti o restart, l'elaborazione dei dati continui senza perdita di informazioni, grazie alla sua capacità di gestire checkpoint e garantire la consistenza dei dati.

[10] (**RF4 – Elaborazione in tempo reale, RF5 – Gestione dei fallimenti, RNF1 – Scalabilità, RNF2 – Performance, RNF3 – Affidabilità**)

5.2.3 MinIO

MinIO è un object storage distribuito compatibile con Amazon Web Services S3 API, e di conseguenza fornisce tutte le principali S3 features. MinIO è progettato per essere distribuito ovunque, anche su cloud pubblici o privati. MinIO si distingue per il fatto che è stato progettato sin dall'inizio per essere lo standard nello storage a oggetti su cloud privati e ibridi. Poiché MinIO è stato costruito appositamente per gestire solo oggetti, la sua architettura a strato singolo raggiunge tutte le funzionalità necessarie senza compromessi. Il risultato è un cloud-native object server che è al contempo performante, scalabile e leggero. Mentre MinIO eccelle nei casi d'uso tradizionali dello object storage come lo storage secondario, il recupero da disastri e l'archiviazione, è particolarmente unico nel superare le sfide legate al machine learning, all'analisi dei dati e ai carichi di lavoro delle applicazioni cloud-native. Le features principali di MinIO sono le seguenti:

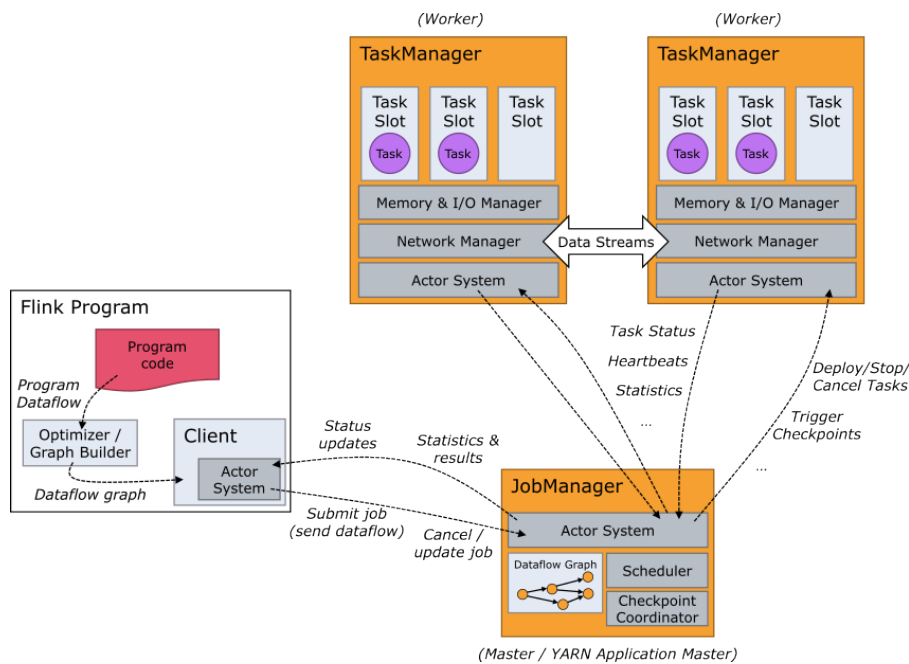


Figura 5.1: Flink Diagram

- **Erasure coding:** un erasure code è definito come un forward error correction (FEC) code basato sull'ipotesi di errori di cancellazione di bit, anziché di errori nei bit. MinIO protegge i dati con per-object inline erasure coding implementato in codice assembly per garantire le massime prestazioni possibili. MinIO utilizza il codice di Reed-Solomon per suddividere gli oggetti in blocchi di dati e parità con livelli di ridondanza configurabili dall'utente. La codifica di cancellazione di MinIO esegue la riparazione a livello di oggetto e può riparare più oggetti in modo indipendente.
- **Bitrot Protection:** Il bitrot è un problema grave che interessa i dischi rigidi, che causa la corruzione dei dati senza che l'utente se ne accorga. Le cause sono molteplici (dischi invecchiati, picchi di corrente, bug nel firmware del disco, scritture fantasma, letture/scritture indirizzate erroneamente, errori nei driver, sovrascritture accidentali), ma il risultato è sempre lo stesso: dei dati compromessi. L'implementazione ottimizzata di MinIO dell'algoritmo HighwayHash garantisce che i dati corrotti non vengano mai letti; invece cattura e ripara gli oggetti corrotti in tempo reale. L'integrità dei dati è assicurata end-to-end attraverso il calcolo di un hash in fase di lettura e la sua verifica in fase di scrittura dall'applicazione.
- **Identity Management:** MinIO supporta i più avanzati standard di gestione dell'identità, integrandosi con i provider compatibili con OpenID Connect e con i principali fornitori di IDP esterni. Ciò significa che l'accesso è centralizzato e le password sono temporanee e ruotano, e non vengono conservate nei file di configurazione o nei database. Inoltre, le politiche di accesso sono molto dettagliate e altamente configurabili, il che rende semplice supportare implementazioni multi-tenant e multi-instance.
- **Continuous Replication:** La sfida con gli approcci di replica tradizionali è che non scalano efficacemente oltre alcune centinaia di TiB. Detto ciò, ogni azienda ha bisogno di una strategia di replica per supportare il disaster recovery e tale strategia deve coprire anche data center e cloud. La replica continua di MinIO è progettata per implementazioni su larga scala e cross-data center. Sfruttando le Lambda compute notifications e i metadati degli oggetti, è in grado di calcolare il delta in modo efficiente e rapido. Le Lambda compute notifications assicurano che le modifiche vengano propagate immediatamente, a differenza della modalità batch tradizionale. La replica continua significa che la perdita di dati sarà ridotta al minimo in caso di guasto, anche in presenza di dataset altamente dinamici. Infine, come tutto ciò che MinIO fa, la replica continua è multi-vendor, il che significa che la tua destinazione di backup può essere qualsiasi cosa, da un NAS al cloud pubblico.

- **Global Federation:** Al giorno d'oggi si hanno dati ovunque. MinIO consente di combinare queste diverse istanze per formare uno spazio dei unified global namespace. In particolare, un numero qualsiasi di server MinIO può essere combinato in un insieme in Modalità Distribuita, e più insiemi in Modalità Distribuita possono essere uniti in una MinIO Server Federation. Ogni MinIO Server Federation fornisce un'amministrazione e uno spazio dei nomi unificati. Un MinIO Server Federation supporta un numero illimitato di insiemi in Modalità Distribuita. L'impatto di questo approccio è che uno storage a oggetti può scalare enormemente per le grandi imprese distribuite geograficamente.

Infine MinIO offre una console web integrata, chiamata **MinIO Console**, che consente di monitorare in tempo reale lo stato del sistema di storage, dove l'utente può visualizzare informazioni dettagliate sui bucket, sugli oggetti salvati, sullo stato dei nodi nel cluster, nonché sulle metriche relative a performance, spazio utilizzato e operazioni recenti. La console espone anche dati relativi alla salute del sistema, come avvisi di errore, segnalazioni di degrado o perdita di disponibilità di dischi. [16] (**RF3 – Archiviazione dei dati su MinIO, RNF1 – Scalabilità, RNF5 – Interoperabilità, RNF6 – Monitoraggio e osservabilità**)

- MinIO Architecture

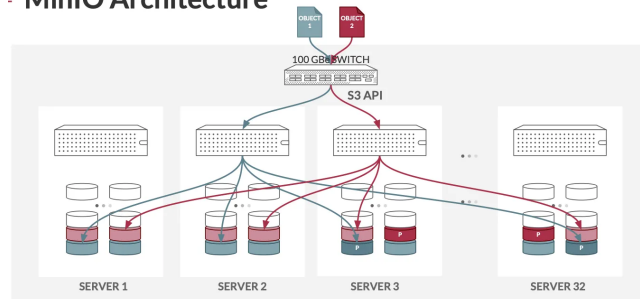


Figura 5.2: MinIO Diagram

5.2.4 Apache Parquet

Parquet è un formato di file open source column-oriented, progettato per una efficiente memorizzazione e recupero dei dati. Offre compressione dei dati e codifica efficienti, con ottime prestazioni nella gestione di grandi volumi di dati. Parquet è un formato di file gratuito e open source, progettato per essere indipendente dal linguaggio, il che significa che può essere utilizzato con diversi linguaggi di programmazione senza problemi di compatibilità. Essendo un formato column-oriented, i dati vengono organizzati per colonna anziché per riga, ottimizzando così lo spazio di archiviazione e migliorando le prestazioni delle query. Inoltre, Parquet offre una compressione dei dati estremamente efficiente, che riduce significativamente lo spazio necessario per memorizzare grandi volumi di informazioni e accelera il processo di decompressione. Un altro vantaggio di Parquet è la sua capacità di supportare tipi di dati complessi e strutture di dati annidate avanzate, rendendolo ideale per applicazioni che richiedono una gestione sofisticata e scalabile dei dati.

Parquet è particolarmente vantaggioso per l'archiviazione di grandi volumi di dati di vario tipo, come tabelle strutturate, immagini, video e documenti. Grazie alla sua compressione altamente efficiente orientata alle colonne, Parquet consente di risparmiare spazio di archiviazione nel cloud, utilizzando schemi di codifica flessibili che ottimizzano colonne contenenti diversi tipi di dati. Un altro vantaggio significativo è l'aumento delle prestazioni e della velocità nell'elaborazione dei dati, grazie a tecniche come il data skipping, che permettono di eseguire query su valori specifici di colonne senza dover leggere l'intera riga, migliorando l'efficienza delle operazioni di lettura, specialmente con grandi quantità di dati. Parquet è implementato utilizzando l'algoritmo record-shredding e assembly, che gestisce strutture dati complesse, e offre metodi di compressione e codifica altamente efficienti. Questo formato è ottimizzato per lavorare con dati complessi in grande quantità e si distingue per l'abilità di eseguire query su colonne selezionate di tabelle molto grandi, riducendo drasticamente le operazioni di I/O.

Lo storage colonnare offre numerosi vantaggi rispetto ai file basati su righe, come i CSV, garantendo una maggiore efficienza nell'elaborazione dei dati. Quando si eseguono query su dati memorizzati

in formato colonnare, è possibile saltare rapidamente i dati non rilevanti, rendendo le query di aggregazione molto più veloci. Questo approccio consente anche risparmi hardware e riduce la latenza nell'accesso ai dati. Apache Parquet, progettato per supportare strutture di dati complesse e annidate, ottimizza il layout dei file per operazioni su grandi volumi di dati, anche di dimensioni gigabyte per singolo file, rendendolo ideale per dataset di grandi dimensioni. Inoltre, Parquet supporta opzioni di compressione flessibili e schemi di codifica efficienti. Poiché i dati all'interno di ogni colonna sono simili tra loro, la compressione delle colonne risulta semplice ed efficace, migliorando ulteriormente le prestazioni delle query. I dati possono essere compressi utilizzando diversi codec disponibili, permettendo una compressione personalizzata in base alle necessità specifiche. [17] (**RF2 – Scrittura dei dati in formato Parquet, RNF2 – Performance, RNF5 – Interoperabilità**)

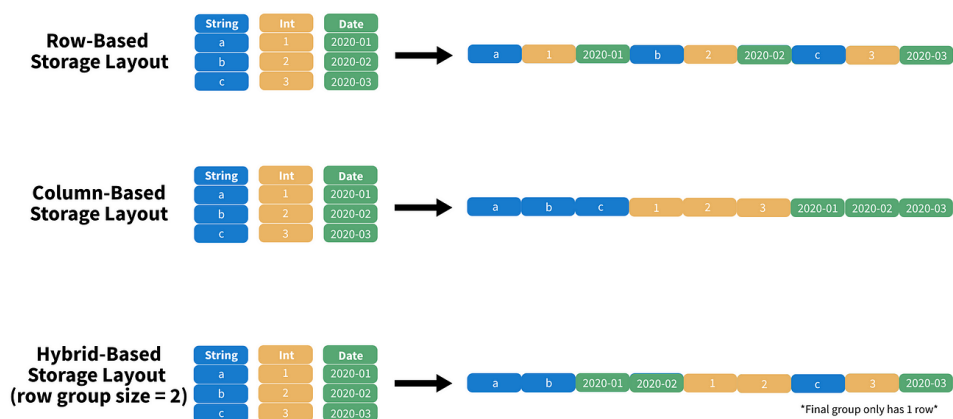


Figura 5.3: Row vs Column Storage

6 Implementazione

6.1 Kafka Delta Sink

Nel codice di esempio proposto, i nomi utilizzati per topic Kafka, gruppi di consumer e altri parametri di configurazione sono stati volutamente mantenuti semplici e generici. Questo per rendere più chiara la logica generale del sistema, senza complicarla con dettagli implementativi legati al contesto specifico. In un ambiente produttivo, questi valori sarebbero invece più descrittivi e strutturati, e in genere configurabili tramite file esterni o variabili d'ambiente, così da garantire maggiore flessibilità. Inoltre, sarebbe normale trovarsi a gestire più topic e più gruppi consumer allo stesso tempo, ciascuno pensato per scopi diversi, come ad esempio separare i flussi in base al tipo di dato, alla fonte o alla priorità.

Il class diagram è il seguente:

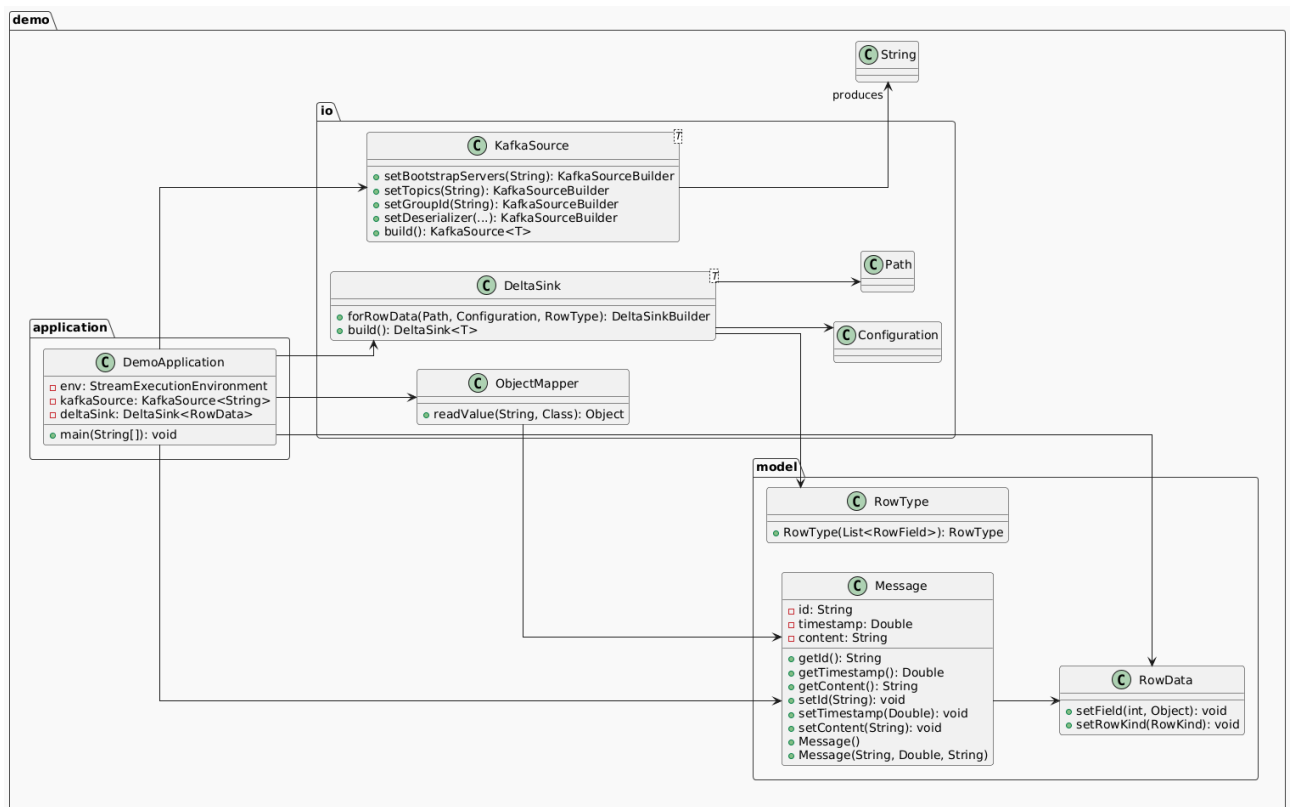


Figura 6.1: Class Diagram

Message Classe POJO (Plain Old Java Object) utilizzata per rappresentare i messaggi provenienti da Kafka. Contiene tre attributi: `id`, `timestamp` e `content`, ed è usata per la deserializzazione dei dati JSON.

DemoApplication Classe principale che contiene il metodo `main`, punto di ingresso dell'applicazione. Si occupa di configurare l'ambiente Flink, inizializzare Kafka come sorgente, trasformare i dati e scriverli nel DeltaSink.

KafkaSource<String> Rappresenta la sorgente di streaming da Apache Kafka. È configurata per leggere i messaggi da un topic specifico e passarli al flusso di elaborazione Flink come stringhe.

DeltaSink<RowData> Componente di uscita che scrive i dati in formato Delta Lake all'interno di un bucket MinIO. Accetta oggetti **RowData**, che corrispondono alla struttura dati trasformata a partire dai messaggi Kafka.

StreamExecutionEnvironment Ambiente di esecuzione fornito da Flink, responsabile dell'orchestrazione del flusso di dati, gestione del checkpointing e avvio del job. È inizializzato in **DemoApplication**.

L'utilizzo combinato di questi componenti garantisce un'elaborazione affidabile e scalabile, coerente con gli scenari tipici di un'architettura moderna per Big Data streaming.

Passando all'implementazione vera e propria, il sistema si basa sull'utilizzo di Apache Flink, che viene impiegato come motore per l'elaborazione dei dati in streaming. L'applicazione è scritta in Java e racchiusa in una classe chiamata **DemoApplication**, che contiene il metodo **main**, da cui parte l'intera esecuzione. In questo metodo viene creato l'ambiente di Flink, e viene subito attivato il checkpointing con un intervallo di 10 secondi. Il checkpointing è fondamentale per evitare la perdita dei dati in caso di guasti: se il job dovesse interrompersi, Flink potrà riprendere dal punto in cui si era fermato.

```
1 final StreamExecutionEnvironment env = StreamExecutionEnvironment.  
    getExecutionEnvironment();  
2 env.enableCheckpointing(10000);
```

Listing 6.1: Inizializzazione dell'ambiente di esecuzione Flink

Dopo l'inizializzazione, viene definita la struttura del messaggio che il sistema andrà a trattare. Lo schema è composto da tre campi: **id**, **timestamp** e **content**, ognuno con il relativo tipo di dato. Questo schema è utile per indicare a Flink e al sink di destinazione come sono organizzati i dati.

```
1 RowType messageType = new RowType(List.of(  
2     new RowType.RowField("id", VarCharType.STRING_TYPE),  
3     new RowType.RowField("timestamp", new DoubleType()),  
4     new RowType.RowField("content", VarCharType.STRING_TYPE)  
5 ));
```

Listing 6.2: Definizione dello schema del messaggio

La sorgente dei dati è un topic Kafka chiamato "my-topic", a cui l'applicazione si connette grazie a un consumer configurato con le impostazioni minime: broker, topic, gruppo consumer, e alcune proprietà relative alla gestione degli offset. La deserializzazione dei messaggi è semplice, in quanto vengono trattati come stringhe.

```
1 KafkaSource<String> kafkaSource = KafkaSource.<String>builder()  
2     .setBootstrapServers("kafka-service:9092")  
3     .setTopics("my-topic")  
4     .setGroupId("my-group")  
5     .setProperty("enable.auto.commit", String.valueOf(true))  
6     .setProperty("auto.commit.interval.ms", String.valueOf(5))  
7     .setStartingOffsets(OffsetsInitializer.committedOffsets(OffsetResetStrategy.  
    LATEST))  
8     .setDeserializer(KafkaRecordDeserializationSchema.valueOnly(StringDeserializer.  
    class));
```



```
9 .build();
```

Listing 6.3: Creazione della sorgente Kafka

Prima di poter scrivere su MinIO, è necessario configurare Hadoop in modo che Flink possa interagire con il bucket S3 compatibile. Questo richiede di specificare l'endpoint del servizio MinIO, insieme a una coppia di credenziali (access key e secret key). Inoltre, viene abilitata la modalità path-style, che è spesso richiesta per compatibilità con MinIO.

```
1 org.apache.hadoop.conf.Configuration hadoopConfig = new org.apache.hadoop.conf.  
  Configuration();  
2 hadoopConfig.set("fs.s3a.endpoint", "http://minio-service:9000");  
3 hadoopConfig.set("fs.s3a.access.key", "miniouser");  
4 hadoopConfig.set("fs.s3a.secret.key", "miniopassword");  
5 hadoopConfig.set("fs.s3a.path.style", "true");
```

Listing 6.4: Configurazione Hadoop per MinIO

Dopo aver impostato la configurazione, si procede alla creazione del DeltaSink, che sarà il componente incaricato di scrivere i dati trasformati nel formato Delta Lake. In questa fase vengono forniti il percorso di destinazione, la configurazione Hadoop e lo schema dei dati. Il blocco è inserito all'interno di un try-catch per intercettare eventuali errori, ad esempio se il bucket non è accessibile o se la configurazione presenta problemi.

```
1 DeltaSink<RowData> deltaSink = null;  
2  
3 try {  
4     deltaSink = DeltaSink  
5         .forRowData(  
6             new Path("s3a://your-bucket-name/table1"),  
7             hadoopConfig,  
8             messageType  
9         )  
10        .build();  
11    System.out.println("DeltaSink creato con successo");  
12 } catch (Exception e) {  
13     System.err.println("Errore nella creazione di DeltaSink: " + e.getMessage());  
14     e.printStackTrace();  
15 }
```

Listing 6.5: Creazione del DeltaSink

Una volta che il sink è stato creato correttamente, viene definita la trasformazione vera e propria del flusso. Ogni messaggio letto da Kafka (sotto forma di stringa JSON) viene deserializzato in un oggetto Java della classe Message, dopodiché viene convertito in un oggetto RowData, che rappresenta il formato richiesto per scrivere su Delta Lake. Durante questa conversione, i singoli campi vengono estratti e mappati secondo lo schema previsto.

```
1 env.fromSource(kafkaSource, WatermarkStrategy.forMonotonousTimestamps(), "Kafka")  
2 .map(element -> {
```

```

3      var ctn = new ObjectMapper().readValue(element, Message.class);
4      var row = new GenericRowData(3);
5      row.setField(0, StringData.fromString(ctn.getId()));
6      row.setField(1, ctn.getTimestamp());
7      row.setField(2, StringData.fromString(ctn.getContent()));
8      row.setRowKind(RowKind.INSERT);
9      System.out.println("Received message: " + element);
10     return (RowData) row;
11 })
12 .sinkTo(deltaSink);

```

Listing 6.6: Trasformazione del flusso e scrittura su Delta Lake

Infine, il flusso viene collegato al sink tramite il metodo `.sinkTo(...)`, e l'intero job viene avviato con `env.execute(...)`, che attiva la pipeline e la mantiene in esecuzione continua.

```

1 env.execute("Kafka to Delta Lake Job on MinIO");

```

Listing 6.7: Esecuzione del job Flink

Nel codice è inclusa anche la definizione della classe `Message`, che è una semplice struttura con tre attributi (`id`, `timestamp` e `content`) e i relativi metodi `getter/setter`. Questa classe serve a gestire più facilmente la deserializzazione dei dati in arrivo da Kafka.

```

1 public class Message {
2     String id;
3     Double timestamp;
4     String content;
5
6     public String getId() { return id; }
7     public Double getTimestamp() { return timestamp; }
8     public String getContent() { return content; }
9
10    public void setId(String id) { this.id = id; }
11    public void setTimestamp(Double timestamp) { this.timestamp = timestamp; }
12    public void setContent(String content) { this.content = content; }
13
14    public Message(String id, Double timestamp, String content) {
15        this.id = id;
16        this.timestamp = timestamp;
17        this.content = content;
18    }
19
20    public Message() {}
21 }

```

Listing 6.8: Classe Message

Nel complesso, questo sistema rappresenta un caso piuttosto realistico di elaborazione dati in streaming, con un'integrazione tra Kafka, Flink e MinIO tramite Delta Lake. Il tutto è pensato per

garantire affidabilità, scalabilità e flessibilità, ed è facilmente estendibile con nuovi topic, trasformazioni o destinazioni dati, se necessario.

7 Conclusioni

In questo progetto, l'obiettivo era migliorare una pipeline capace di gestire flussi di dati in tempo reale e conservarli in modo affidabile. Per raggiungere questo scopo, sono state integrate diverse componenti tra loro: Apache Kafka per la gestione dei messaggi, Apache Flink per l'elaborazione degli stream, Delta Lake come formato di scrittura e MinIO come sistema di storage compatibile con S3.

In sostanza, ogni messaggio che arrivava su un topic Kafka veniva intercettato da un job Flink. Dopo essere stato deserializzato, il messaggio veniva trasformato in un oggetto `RowData`, per poi essere scritto in formato Parquet all'interno di un bucket MinIO tramite il sink Delta Lake.

Durante la fase di sviluppo, è stato definito in modo esplicito la sorgente Kafka (`KafkaSource`), impostando uno schema coerente con la tabella di destinazione, e costruito un `DeltaSink<RowData>` tramite `l'API forRowData()`. Quest'ultima è stata configurata per funzionare con file system compatibili con S3A. Si è anche attivato il checkpointing di Flink, essenziale per gestire correttamente lo stato del job e garantire la scrittura dei dati una sola volta, senza duplicazioni.

Tuttavia, una volta avviato il sistema, è emerso un problema piuttosto ostico: i checkpoint partivano ma non si completavano mai. Questo impediva di fatto il funzionamento del sink, poiché la fase di commit dei dati avviene solo a checkpoint completato. Dai log e dalle metriche non risultavano errori evidenti; Flink eseguiva tutte le fasi previste, ma rimaneva bloccato in attesa della conclusione del checkpoint, che non arrivava mai.

Abbiamo controllato che le versioni dei vari componenti fossero compatibili tra loro, come indicato nella documentazione ufficiale, e ci siamo assicurati che le impostazioni del file system fossero corrette. Dopo varie prove, è emerso che l'API che stavamo utilizzando per creare il sink risultava deprecata nelle versioni più recenti del connettore Delta per Flink.

Questa scoperta ci ha portato a ipotizzare che il comportamento anomalo potesse essere causato da modifiche introdotte nel connettore, che probabilmente non gestisce più i checkpoint nello stesso modo o richiede configurazioni specifiche non documentate.

Nel complesso, la struttura del sistema era solida e aderente ai requisiti progettuali, ma a causa di queste difficoltà legate al checkpointing e all'uso di API non più supportate, non è stato possibile completare con successo la fase di validazione finale. Per affrontare il problema in futuro, si potrebbe considerare l'adozione di versioni più aggiornate del connettore, compatibili con Flink 1.17 o superiori. Un'altra possibilità sarebbe utilizzare componenti alternativi, come `FileSystemLogSink`, che potrebbero offrire maggiore stabilità. Infine, strumenti di orchestrazione e monitoraggio più avanzati aiuterebbero a isolare meglio i punti critici della pipeline, aumentando così affidabilità e osservabilità.

Bibliografia

- [1] Ivan Kayongo, Leonardo Malcotti, Haonan Zhao, and Fausto Giunchiglia. A methodology and a platform for high-quality rich personal data. <https://arxiv.org/abs/2501.16864>, 2025.
- [2] Big data thick data. <https://profvkshrivastava.medium.com/is-big-data-vs-thick-data-or-big-data-complements-thick-data-d752c127703a>.
- [3] Data warehouse ibm. <https://www.ibm.com/think/topics/data-warehouse>.
- [4] Data warehouse oracle. <https://www.oracle.com/database/what-is-a-data-warehouse/>.
- [5] Hadoop. <https://learn.microsoft.com/en-us/azure/hdinsight/hadoop/apache-hadoop-introduction>.
- [6] Hdfs. <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>.
- [7] Aws. <https://neal-davis.medium.com/the-history-of-aws-and-the-evolution-of-computing-0a64cee5bc15>.
- [8] Delta. <https://docs.delta.io/latest/index.html>.
- [9] Delta lake. <https://business.adobe.com/blog/basics/what-is-a-data-lake>.
- [10] Apache flink. <https://nightlies.apache.org/flink/flink-docs-master/>.
- [11] Apache kafka. <https://kafka.apache.org/documentation/>.
- [12] Lambda architecture. <https://www.databricks.com/it/glossary/lambda-architecture>.
- [13] Delta architecture. <https://www.databricks.com/blog/2020/11/20/delta-vs-lambda-why-simplicity-trumps-complexity-for-data-pipelines.html>.
- [14] Lakehouse. <https://www.databricks.com/it/glossary/data-lakehouse>.
- [15] Kubernetes. <https://kubernetes.io/docs/home/>.
- [16] Minio. <https://min.io/product/overview>.
- [17] Apache parquet. <https://www.databricks.com/glossary/what-is-parquet>.